ΕΡΓΑΣΤΗΡΙΟ 4 · ΕΝΟΤΗΤΑ 2.1.Β.2 · ΤΡΙΤΗ 03/11/2026

Αλγόριθμοι που διακόπτουν

Χρονοδρομολόγηση II: Round Robin, SRTF και Priority

Θεωρία σε 10 μικρά βήματα + 16 ασκήσεις με λύσεις βήμα-βήμα · καυτή πατάτα · Excel · 37 διαφάνειες

Λαμπρόπουλος Χρήστος

Οι 16 ασκήσεις του εργαστηρίου

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
1	Με ή χωρίς διακοπή;	Εύκολο	3 → 4
2	Η καυτή πατάτα	Εύκολο	8 → 9
3	Round Robin με κβάντο 2	Εύκολο	10 → 11
4	SRTF: το πρώτο παράδειγμα	Εύκολο	14 → 15
5	SRTF με 4 διεργασίες	Μέτριο	16 → 17
6	Priority χωρίς διακοπή	Μέτριο	19 → 20
7	Priority με διακοπή	Ο στόχος	22 → 23
8	Ποιος κερδίζει;	Δύσκολο	24 → 25

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
9	Κουίζ επανάληψης	Μέτριο	26 → 27
10	Πολύ μεγάλο κβάντο	Για το σπίτι	28 → 29
11	nice: η ευγενική διεργασία	Εύκολο	30 → 31
12	renice: χαμήλωσε το «φορτηγό»	Μέτριο	30 → 31
13	start με προτεραιότητα	Εύκολο	32 → 33
14	Οι προτεραιότητες του συστήματος	Μέτριο	32 → 33
15	Πάγωσε μια ολόκληρη ουρά	Μέτριο	34 → 35
16	Ταξινόμησε κατά προτεραιότητα	Δύσκολο	34 → 35

Στήλη «Διαφάνεια»: **εκφώνηση** → **λύση**.

ΠΩΣ Διαβάζεις το βήμα → λύνεις στο site την άσκηση της ετικέτας **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** → ελέγχεις με τη **λύση** που ακολουθεί.

Μπορεί το ΛΣ να τη σταματήσει;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 1

Η ΕΡΩΤΗΣΗ ΤΗΣ ΗΜΕΡΑΣ

Μια διεργασία κρατά τον επεξεργαστή **10 λεπτά** κι εσύ θέλεις απλώς να κουνήσεις το ποντίκι. Με FCFS ή SJF περιμένεις να τελειώσει! Γι' αυτό τα σημερινά ΛΣ χρησιμοποιούν αλγορίθμους που **διακόπτουν**.

Χωρίς διακοπή · μη προεκχωρητικοί

FCFS · **SJF** · **Priority** (απλή μορφή)
Όποια διεργασία ξεκινήσει, **τελειώνει**. (Εργαστήριο 3)

Με διακοπή · προεκχωρητικοί (preemptive)

Round Robin: σταματά όταν λήξει το **κβάντο**
SRTF: σταματά αν έρθει **πιο σύντομη**
Priority με διακοπή: σταματά αν έρθει **πιο σημαντική**

ΔΙΑΚΟΠΗ Η διεργασία που σταματά **δεν χάνεται**: γυρνά στην ουρά και συνεχίζει αργότερα από εκεί που έμεινε.

Με ή χωρίς διακοπή: λύση

ΕΡΏΤΗΣΗ	ΣΩΣΤΉ ΑΠΆΝΤΗΣΗ	ΓΙΑΤΊ
FCFS	Χωρίς διακοπή	Με τη σειρά άφιξης, ως το τέλος.
SJF	Χωρίς διακοπή	Διαλέγει τη σύντομη **μόνο** όταν αδειάσει ο επεξεργαστής.
Round Robin	Με διακοπή	Σταματά τη διεργασία όταν λήξει το **κβάντο** της.
SRTF	Με διακοπή	Είναι το SJF **με διακοπή**: σταματά αν έρθει πιο σύντομη.
Priority (απλή μορφή)	Χωρίς διακοπή	Η πιο σημαντική περιμένει να τελειώσει αυτή που τρέχει.
Priority με διακοπή	Με διακοπή	Η πιο σημαντική **σταματά** αμέσως αυτή που τρέχει.
Ο αλγόριθμος που σταματά διεργασία στη μέση λέγεται…	προεκχωρητικός	Preemptive: «παίρνει πίσω» τον επεξεργαστή.

ΘΥΜΗΣΟΥ Χωρίς διακοπή = **μη προεκχωρητικός** · με διακοπή = **προεκχωρητικός**.

Παιχνίδι: η καυτή πατάτα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 2

ΤΙ ΚΑΝΟΥΜΕ

Ένα μπαλάκι είναι ο **επεξεργαστής**. 3 εκπαιδευόμενοι είναι **διεργασίες** με κάρτες **5, 3 και 1** δευτερολέπτων. Το **κβάντο** είναι 2 δευτερόλεπτα. Ένας μετράει δυνατά, ο εκπαιδευτής γράφει στον πίνακα.

1 · Κρατάς το μπαλάκι 2΄΄

Όποιος έχει το μπαλάκι «τρέχει» και αφαιρεί **2** από την κάρτα του (ή ό,τι του έμεινε).

2 · Το δίνεις στον επόμενο

Αν σου έμειναν δευτερόλεπτα, πας στο **τέλος της ουράς** και περιμένεις ξανά τη σειρά σου.

3 · Τελείωσες;

Κάρτα = 0: κάθεσαι και ο εκπαιδευτής γράφει **πότε τελείωσες**.

Για τον εκπαιδευτή

Σειρά: P1 → P2 → P3 → P1 → P2 → P1. Τελειώνουν: **P3 στο 5, P2 στο 8, P1 στο 9**.

ΜΕΤΑ Ζωγράφισε στο site (**άσκηση 2**) τι έγινε στο παιχνίδι. Πρώτα δες τα βήματα 2 και 3.

Round Robin: εκ περιτροπής

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 2 · 3

Η ΙΔΕΑ

Κάθε διεργασία παίρνει τον επεξεργαστή για ένα **κβάντο** χρόνου. Αν δεν τελειώσει, πάει στο **τέλος της ουράς**. Όλες παίρνουν σειρά: είναι **δίκαιος**.

ΠΑΡΑΔΕΙΓΜΑ · P1 = 3, P2 = 4, P3 = 2 · ΟΛΕΣ ΣΤΟ 0 · ΚΒΑΝΤΟ = 2



Ο κύκλος

P1 2 → P2 2 → P3 2 (**τελείωσε**) → P1 1 (**τελείωσε**: δεν χρειάζεται όλο το κβάντο) → P2 2 (**τελείωσε**).

Ο κανόνας της ουράς

Αν τη στιγμή που λήγει ένα κβάντο **φτάνει** και νέα διεργασία, μπαίνει **πρώτα η νέα** και μετά αυτή που διακόπηκε.

ΚΑΝΟΝΑΣ Τρέχει πάντα η **πρώτη** της ουράς, για **το πολύ** ένα κβάντο.

Πώς μετράω την αναμονή τώρα;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 2 · 3

ΤΟ ΠΡΟΒΛΗΜΑ

Με διακοπή μια διεργασία περιμένει **πολλές φορές**, σε κομμάτια. Το «αρχίζει − άφιξη» δεν φτάνει: η P1 αρχίζει στο 0, αλλά μετά περιμένει από το 2 ως το 6.



ΔΙΕΡΓΑΣΊΑ	ΚΟΜΜΆΤΙΑ ΑΝΑΜΟΝΉΣ	ΤΈΛΟΣ − ΆΦΙΞΗ − ΕΚΤΈΛΕΣΗ	ΑΝΑΜΟΝΉ
P1	(2–6) 4	7 − 0 − 3	**4**
P2	(0–2) 2 + (4–7) 3	9 − 0 − 4	**5**
P3	(0–4) 4	6 − 0 − 2	**4**

ΤΥΠΟΣ Αναμονή = **τέλος − άφιξη − εκτέλεση**: όλη η ώρα στο σύστημα μείον την ώρα που έτρεχε. Μέσος όρος (4 + 5 + 4) / 3 = **4,33**.

Η καυτή πατάτα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 2

ΕΚΦΩΝΗΣΗ

Ζωγράφισε το παιχνίδι: κάρτες **5, 3 και 1**, όλες στο 0, κβάντο **2**. Σειρά P1 → P2 → P3 και ξανά από την αρχή, μόνο όσες δεν τελείωσαν.

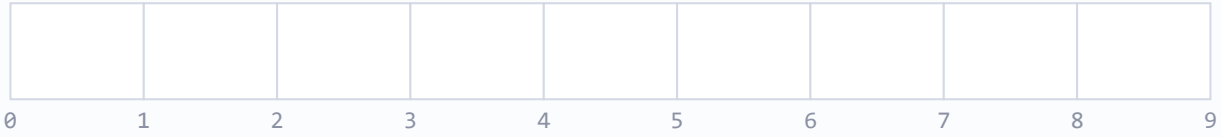
ΔΕΔΟΜΕΝΑ · ΚΒΑΝΤΟ = 2

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	5
P2	0	3
P3	0	1

Μέσος χρόνος αναμονής

(___ + ___ + ___) / 3 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		
P3		

ΣΤΟ SITE Με διακοπή ο πίνακας **δεν** έχει «Αρχίζει»: συμπληρώνεις μόνο **τέλος** και **αναμονή**.

Η καυτή πατάτα: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΟΥΡΆ (ΠΡΏΤΗ →)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1, P2, P3	P1 (0–2)	Τρέχει η **P1**, η πρώτη της ουράς
2	P2, P3, P1	P2 (2–4)	Έληξε το κβάντο της P1: πάει στο **τέλος** της ουράς → τρέχει η **P2**, η πρώτη της ουράς
4	P3, P1, P2	P3 (4–5)	Έληξε το κβάντο της P2: πάει στο **τέλος** της ουράς → τρέχει η **P3**, η πρώτη της ουράς
5	P1, P2	P1 (5–7)	Τελείωσε η P3 → τρέχει η **P1**, η πρώτη της ουράς
7	P2, P1	P2 (7–8)	Έληξε το κβάντο της P1: πάει στο **τέλος** της ουράς → τρέχει η **P2**, η πρώτη της ουράς
8	P1	P1 (8–9)	Τελείωσε η P2 → τρέχει η **P1**, η πρώτη της ουράς

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	9 − 0 − 5	**4**
P2	8 − 0 − 3	**5**
P3	5 − 0 − 1	**4**

Μέσος όρος

(4 + 5 + 4) / 3 = 13 / 3 = **4,33**
Η P3 χρειάζεται μόνο 1, αλλά περιμένει 4: πρώτα παίρνουν σειρά οι P1 και P2.

ΘΥΜΗΣΟΥ Αν μια διεργασία τελειώσει **πριν** λήξει το κβάντο, ο επεξεργαστής πάει **αμέσως** στην επόμενη (η P3 στο 4–5).

Round Robin με κβάντο 2

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 3

ΕΚΦΩΝΗΣΗ

Όλες φτάνουν στο **0**, με σειρά P1, P2, P3. Κβάντο = **2**: κάθε διεργασία τρέχει το πολύ 2 κουτάκια και, αν δεν τελειώσει, πηγαίνει στο τέλος της ουράς.

ΔΕΔΟΜΕΝΑ · ΚΒΑΝΤΟ = 2

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	4
P2	0	2
P3	0	3

Μέσος χρόνος αναμονής

(___ + ___ + ___) / 3 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		
P3		

ΣΤΟ SITE P1 στο 0–2, P2 στο 2–4 (και **τελειώνει**), P3 στο 4–6… και συνεχίζεις κυκλικά.

Round Robin με κβάντο 2: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΟΥΡΆ (ΠΡΏΤΗ →)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1, P2, P3	P1 (0–2)	Τρέχει η **P1**, η πρώτη της ουράς
2	P2, P3, P1	P2 (2–4)	Έληξε το κβάντο της P1: πάει στο **τέλος** της ουράς → τρέχει η **P2**, η πρώτη της ουράς
4	P3, P1	P3 (4–6)	Τελείωσε η P2 → τρέχει η **P3**, η πρώτη της ουράς
6	P1, P3	P1 (6–8)	Έληξε το κβάντο της P3: πάει στο **τέλος** της ουράς → τρέχει η **P1**, η πρώτη της ουράς
8	P3	P3 (8–9)	Τελείωσε η P1 → τρέχει η **P3**, η πρώτη της ουράς

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	8 − 0 − 4	**4**
P2	4 − 0 − 2	**2**
P3	9 − 0 − 3	**6**

Μέσος όρος

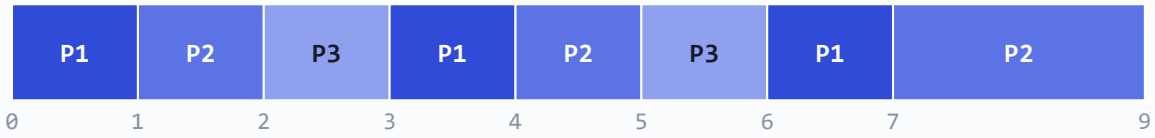
(4 + 2 + 6) / 3 = 12 / 3 = **4**
Η P2 (2) χωράει σε ένα κβάντο: τελειώνει με την πρώτη της σειρά.

ΘΥΜΗΣΟΥ Αναμονή P3 = 9 − 0 − 3 = **6**: περίμενε 0–4 και 6–8. Ο τύπος βγάζει το ίδιο με τα κομμάτια.

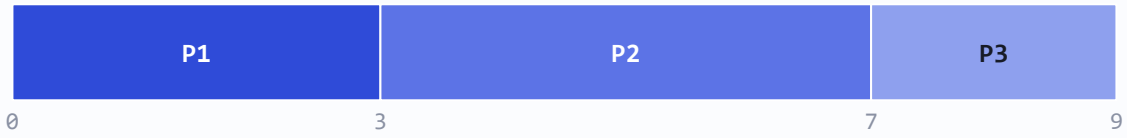
Μικρό ή μεγάλο κβάντο;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 10

ΤΟ ΙΔΙΟ ΠΑΡΑΔΕΙΓΜΑ · ΚΒΑΝΤΟ = 1 · 7 ΕΝΑΛΛΑΓΕΣ



ΤΟ ΙΔΙΟ ΠΑΡΑΔΕΙΓΜΑ · ΚΒΑΝΤΟ = 5 · 2 ΕΝΑΛΛΑΓΕΣ



Πολύ μικρό κβάντο

Κάθε αλλαγή διεργασίας (**εναλλαγή**, context switch) θέλει χρόνο: το ΛΣ σώζει πού έμεινε η μία και φορτώνει την άλλη. Πολλές εναλλαγές = **χαμένος χρόνος**. Μέσος όρος εδώ: 4,33.

Πολύ μεγάλο κβάντο

Καμία δεν χρειάζεται δεύτερη σειρά: ο Round Robin γίνεται **FCFS** (και ξαναβλέπουμε το φαινόμενο συνοδείας). Μέσος όρος εδώ: 3,33.

ΣΤΗΝ ΠΡΑΞΗ Το κβάντο είναι λίγα χιλιοστά του δευτερολέπτου (π.χ. **10–100 ms**): αρκετό για να μη χάνεται χρόνος, μικρό για να «πετάει» το ποντίκι.

SRTF: όποια είναι πιο κοντά στο τέλος

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 4 · 5

Η ΙΔΕΑ

Είναι το **SJF με διακοπή**. Κάθε φορά που **φτάνει** νέα διεργασία, το ΛΣ κοιτάζει πόσος χρόνος **λείπει** από την καθεμία (υπόλοιπος χρόνος). Τρέχει όποια έχει τον **μικρότερο υπόλοιπο**.

ΠΑΡΑΔΕΙΓΜΑ · P1 (ΑΦΙΞΗ 0, ΕΚΤΕΛΕΣΗ 4) · P2 (1, 1) · P3 (2, 3)



Τι έγινε στο 1;

Ήρθε η P2 (θέλει **1**). Στην P1 λείπουν ακόμα **3**. 1 < 3, άρα η P1 **σταματά** και τρέχει η P2.

Ισοπαλία στο 2

P1 και P3 θέλουν από **3**: πάει όποια **ήρθε πρώτη** (P1). Αν μία ήδη τρέχει, **συνεχίζει** αυτή.

ΑΝΑΜΟΝΕΣ P1 = 1, P2 = 0, P3 = 3 → μέσος όρος (1 + 0 + 3) / 3 = **1,33**. Τύπος: **τέλος − άφιξη − εκτέλεση**.

SRTF: το πρώτο παράδειγμα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 4

ΕΚΦΩΝΗΣΗ

Η P1 ξεκινά μόνη της. Στο **2** φτάνει η P2, που χρειάζεται μόνο **2**. Στην P1 λείπουν ακόμα **4**. Ποια συνεχίζει;

ΔΕΔΟΜΕΝΑ

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	6
P2	2	2

Μέσος χρόνος αναμονής

(___ + ___) / 2 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		

ΣΤΟ SITE Το SRTF τρέχει όποια έχει τον **μικρότερο υπόλοιπο**: 2 < 4.

SRTF: το πρώτο παράδειγμα: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΠΕΡΙΜΈΝΟΥΝ (ΥΠΌΛΟΙΠΟ)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1 (6)	P1 (0–2)	Είναι **μόνη** της
2	P1 (4), P2 (2)	P2 (2–4)	Ήρθε η P2 → **διακοπή**: P2 (2) < P1 (4)
4	P1 (4)	P1 (4–8)	Τελείωσε η P2 → η **τελευταία** που έμεινε

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	8 − 0 − 6	**2**
P2	4 − 2 − 2	**0**

Μέσος όρος

(2 + 0) / 2 = 2 / 2 = **1**
Με SJF (χωρίς διακοπή) η P2 θα περίμενε ως το 6: αναμονή 4 αντί για **0**.

ΘΥΜΗΣΟΥ Η P1 κόβεται σε **δύο κομμάτια** (0–2 και 4–8). Περίμενε μόνο 2–4, άρα αναμονή **2**.

SRTF με 4 διεργασίες

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 5

ΕΚΦΩΝΗΣΗ

Κάθε φορά που φτάνει νέα διεργασία (στο **2**, στο **4**, στο **5**), σύγκρινε τον **υπόλοιπο** χρόνο όλων. Τρέχει η μικρότερη. Τα ίδια δεδομένα έλυσες με SJF στην άσκηση 9 του Εργαστηρίου 3.

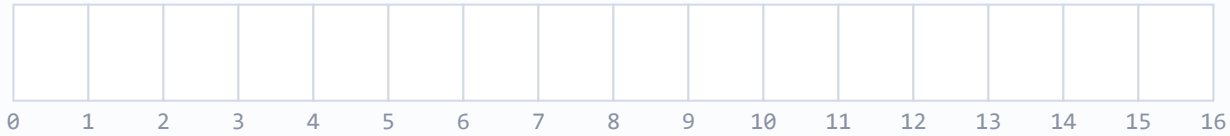
ΔΕΔΟΜΕΝΑ

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Μέσος χρόνος αναμονής

(___ + ___ + ___ + ___) / 4 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		
P3		
P4		

ΣΤΟ SITE Κράτα σε χαρτί τον υπόλοιπο χρόνο κάθε διεργασίας. Σε **ισοπαλία** συνεχίζει αυτή που ήδη τρέχει.

SRTF με 4 διεργασίες: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΠΕΡΙΜΈΝΟΥΝ (ΥΠΌΛΟΙΠΟ)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1 (7)	P1 (0–2)	Είναι **μόνη** της
2	P1 (5), P2 (4)	P2 (2–4)	Ήρθε η P2 → **διακοπή**: P2 (4) < P1 (5)
4	P1 (5), P2 (2), P3 (1)	P3 (4–5)	Ήρθε η P3 → **διακοπή**: P3 (1) < P2 (2)
5	P1 (5), P2 (2), P4 (4)	P2 (5–7)	Τελείωσε η P3. Ήρθε η P4 → η **μικρότερη**: P2 (2)
7	P1 (5), P4 (4)	P4 (7–11)	Τελείωσε η P2 → η **μικρότερη**: P4 (4)
11	P1 (5)	P1 (11–16)	Τελείωσε η P4 → η **τελευταία** που έμεινε

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	16 − 0 − 7	**9**
P2	7 − 2 − 4	**1**
P3	5 − 4 − 1	**0**
P4	11 − 5 − 4	**2**

Μέσος όρος

(9 + 1 + 0 + 2) / 4 = 12 / 4 = **3**
Με SJF ήταν **4**. Η διακοπή της μεγάλης P1 βοήθησε όλες τις μικρές.

ΘΥΜΗΣΟΥ Στο 5 τελειώνει η P3 και φτάνει η P4: περιμένουν P1 (5), P2 (2), P4 (4). Τρέχει η **P2**, γιατί της λείπουν μόνο 2.

Priority: πρώτα η πιο σημαντική

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 6

Η ΙΔΕΑ

Κάθε διεργασία έχει έναν **αριθμό προτεραιότητας**. Όταν ελευθερωθεί ο επεξεργαστής, παίρνει η **πιο σημαντική** από όσες περιμένουν.

ΠΑΡΑΔΕΙΓΜΑ · ΟΛΕΣ ΣΤΟ 0 · P1 (ΕΚΤΕΛΕΣΗ 3, ΠΡΟΤΕΡ. 2) · P2 (2, 1) · P3 (1, 3)



Ο αριθμός

Στις ασκήσεις μας **1 = πιο σημαντική**, όπως στο Linux. Στα Windows ισχύει το **ανάποδο** (βήμα 9).

Χωρίς διακοπή

Όποια ξεκινήσει **τελειώνει**, ακόμα κι αν έρθει πιο σημαντική.
Αναμονή = αρχίζει − άφιξη: P1 = 2, P2 = 0, P3 = 5 → **2,33**

ΚΑΝΟΝΑΣ Όταν φτάνουν όλες μαζί: ταξινόμησε κατά **προτεραιότητα** (1, 2, 3 …) και μετά λύνεις όπως στο FCFS.

Priority χωρίς διακοπή

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 6

ΕΚΦΩΝΗΣΗ

Όλες φτάνουν στο **0**. Τρέχουν με σειρά προτεραιότητας (**1 = πιο σημαντική**). Καμία δεν διακόπτεται.

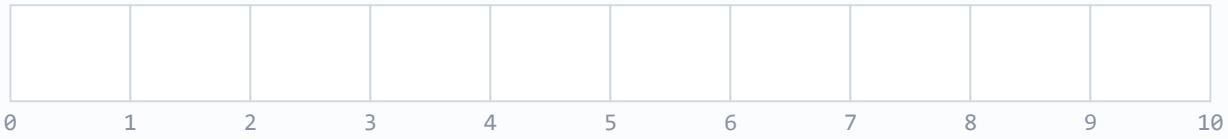
ΔΕΔΟΜΕΝΑ · 1 = ΠΙΟ ΣΗΜΑΝΤΙΚΗ

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ	ΠΡΟΤΕΡ.
P1	0	4	3
P2	0	3	1
P3	0	2	4
P4	0	1	2

Μέσος χρόνος αναμονής

(___ + ___ + ___ + ___) / 4 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΑΡΧΊΖΕΙ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1			
P2			
P3			
P4			

ΣΤΟ SITE Πρώτα η 1, μετά η 2 … Αναμονή = **αρχίζει − άφιξη**.

Priority χωρίς διακοπή: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΑΔΕΙΑΖΕΙ Ο ΕΠΕΞΕΡΓΑΣΤΗΣ

ΣΤΙΓΜΉ	ΠΕΡΙΜΈΝΟΥΝ (ΠΡΟΤΕΡ.)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1 (3), P2 (1), P3 (4), P4 (2)	P2 (0–3)	Η **πιο σημαντική**: P2 (1)
3	P1 (3), P3 (4), P4 (2)	P4 (3–4)	Τελείωσε η P2 → η **πιο σημαντική**: P4 (2)
4	P1 (3), P3 (4)	P1 (4–8)	Τελείωσε η P4 → η **πιο σημαντική**: P1 (3)
8	P3 (4)	P3 (8–10)	Τελείωσε η P1 → η **τελευταία** που έμεινε

ΑΝΑΜΟΝΗ = ΑΡΧΙΖΕΙ − ΑΦΙΞΗ

ΔΙΕΡΓ.	ΑΡΧΊΖΕΙ − ΆΦΙΞΗ	ΑΝΑΜ.
P1	4 − 0	**4**
P2	0 − 0	**0**
P3	8 − 0	**8**
P4	3 − 0	**3**

Μέσος όρος

(4 + 0 + 8 + 3) / 4 = 15 / 4 = **3,75**
Η P3 είναι η λιγότερο σημαντική (4): περιμένει **8** για να τρέξει μόλις 2.

ΘΥΜΗΣΟΥ Η σειρά βγαίνει από τη στήλη **Προτερ.** (P2 → P4 → P1 → P3), **όχι** από τον χρόνο εκτέλεσης.

Όταν έρχεται πιο σημαντική

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 7

Η ΙΔΕΑ

Αν φτάσει **πιο σημαντική** διεργασία, αυτή που τρέχει **σταματά αμέσως** και γυρνά στην ουρά.

ΠΑΡΑΔΕΙΓΜΑ · P1 (ΑΦΙΞΗ 0, ΕΚΤΕΛΕΣΗ 4, ΠΡΟΤΕΡ. 2) · P2 (1, 2, 1)



Τι έγινε στο 1;

Ήρθε η P2 με προτεραιότητα **1**, πιο σημαντική από την P1 (**2**): η P1 σταματά και συνεχίζει στο 3.
P1 = 2, P2 = 0 → **1**

Λιμοκτονία και γήρανση

Αν έρχονται συνέχεια σημαντικές, μια ασήμαντη μπορεί να **μην τρέξει ποτέ** (λιμοκτονία). Λύση: η **γήρανση** (aging). Όσο περιμένει, η προτεραιότητά της **ανεβαίνει** σιγά σιγά.

ΘΥΜΗΣΟΥ Με διακοπή η αναμονή είναι πάλι **τέλος − άφιξη − εκτέλεση**.

Priority με διακοπή

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 7

ΕΚΦΩΝΗΣΗ

Αν φτάσει πιο σημαντική διεργασία, η τρέχουσα **σταματά** και περιμένει να ξανάρθει η σειρά της. Αναμονή = τέλος − άφιξη − εκτέλεση.

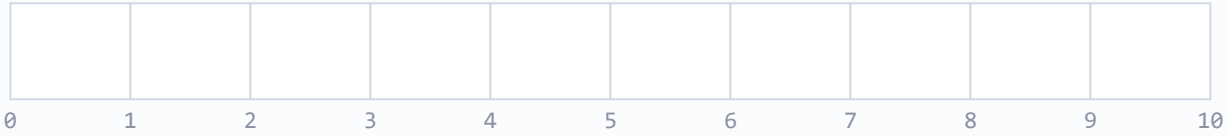
ΔΕΔΟΜΕΝΑ · 1 = ΠΙΟ ΣΗΜΑΝΤΙΚΗ

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ	ΠΡΟΤΕΡ.
P1	0	5	3
P2	1	2	1
P3	2	3	2

Μέσος χρόνος αναμονής

(___ + ___ + ___) / 3 = ___

ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		
P3		

ΣΤΟ SITE Στο 1 φτάνει η P2 (**1**), πιο σημαντική από την P1 (**3**). Στο 2 φτάνει η P3 (**2**): είναι πιο σημαντική από την P2;

Priority με διακοπή: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΠΕΡΙΜΈΝΟΥΝ (ΠΡΟΤΕΡ.)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1 (3)	P1 (0–1)	Είναι **μόνη** της
1	P1 (3), P2 (1)	P2 (1–3)	Ήρθε η P2 → **διακοπή**: P2 (1) < P1 (3)
2	P1 (3), P2 (1), P3 (2)	P2 (2–3)	Ήρθε η P3 → η P2 (1) **συνεχίζει**: είναι ακόμα η πιο σημαντική
3	P1 (3), P3 (2)	P3 (3–6)	Τελείωσε η P2 → η **πιο σημαντική**: P3 (2)
6	P1 (3)	P1 (6–10)	Τελείωσε η P3 → η **τελευταία** που έμεινε

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	10 − 0 − 5	**5**
P2	3 − 1 − 2	**0**
P3	6 − 2 − 3	**1**

Μέσος όρος

(5 + 0 + 1) / 3 = 6 / 3 = **2**
Η P1 (3) είναι η λιγότερο σημαντική: σταματά στο 1 και ξαναπαίρνει τον επεξεργαστή μόνο στο **6**.

ΘΥΜΗΣΟΥ Στο 2 η P3 (2) **δεν** διακόπτει την P2 (1): διακόπτει μόνο η **πιο** σημαντική.

Ποιος κερδίζει;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 8

ΕΚΦΩΝΗΣΗ

Τα δεδομένα της **άσκησης 5** με **τέσσερις** αλγορίθμους. Λύσε τους στο χαρτί ή στο Excel και γράψε τον μέσο χρόνο αναμονής του καθενός.

ΔΕΔΟΜΕΝΑ

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Έχεις ήδη λύσει τα δύο

SJF: άσκηση 9 του Εργαστηρίου 3 · **SRTF**: άσκηση 5.

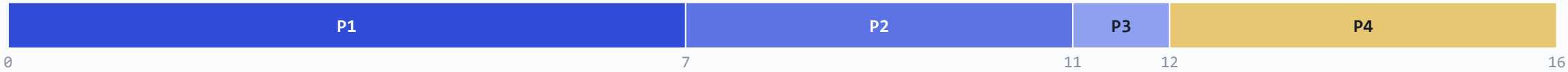
ΜΕΣΟΣ ΧΡΟΝΟΣ ΑΝΑΜΟΝΗΣ

ΑΛΓΌΡΙΘΜΟΣ	ΜΈΣΟΣ ΌΡΟΣ
FCFS	___
SJF	___
SRTF	___
Round Robin (κβάντο 2)	___
Ποιος κερδίζει;	___

ROUND ROBIN Όποια φτάνει μπαίνει στην ουρά **πριν** από αυτήν που μόλις διακόπηκε (βήμα 2).

Ποιος κερδίζει: λύση

FCFS · ΑΝΑΜΟΝΕΣ 0, 5, 7, 7 · ΜΕΣΟΣ ΟΡΟΣ 4,75



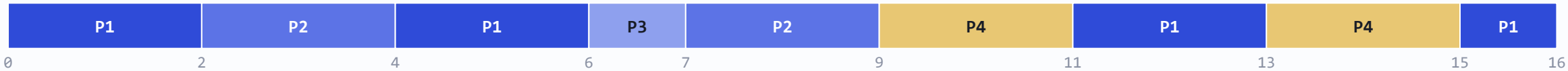
SJF · ΑΝΑΜΟΝΕΣ 0, 6, 3, 7 · ΜΕΣΟΣ ΟΡΟΣ 4



SRTF · ΑΝΑΜΟΝΕΣ 9, 1, 0, 2 · ΜΕΣΟΣ ΟΡΟΣ 3



ROUND ROBIN (ΚΒΑΝΤΟ 2) · ΑΝΑΜΟΝΕΣ 9, 3, 2, 6 · ΜΕΣΟΣ ΟΡΟΣ 5



Νικητής: SRTF (3)

Θέλει όμως να **ξέρει από πριν** πόσο θα κρατήσει κάθε διεργασία. Στην πράξη το ΛΣ μόνο το **μαντεύει**.

Και ο Round Robin (5);

Δεν κερδίζει σε αναμονή, αλλά **καμία** διεργασία δεν κρατά τον επεξεργαστή για πολύ: γι' αυτό τον θέλουν τα ΛΣ.

ΣΕΙΡΑ SRTF **3** < SJF **4** < FCFS **4,75** < Round Robin **5**.

Κουίζ επανάληψης

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 9

ΕΡΩΤΗΣΗ	ΕΠΙΛΟΓΕΣ
Ποιος αλγόριθμος εξυπηρετεί με σειρά άφιξης;	FCFS · SJF · Round Robin · SRTF
Ποιος δίνει τον μικρότερο μέσο χρόνο αναμονής χωρίς διακοπή;	FCFS · SJF · Round Robin · SRTF
Τι είναι το κβάντο χρόνου;	ο χρόνος κάθε σειράς στον RR · η συνολική εκτέλεση · η αναμονή
Ποιοι αλγόριθμοι διακόπτουν;	FCFS και SJF · RR, SRTF, Priority με διακοπή · μόνο ο RR
Τι είναι η λιμοκτονία;	δεν παίρνει ποτέ επεξεργαστή · άδειος επεξεργαστής · γεμάτη μνήμη
Ποια λύση υπάρχει για τη λιμοκτονία;	η γήρανση (aging) · μεγαλύτερο κβάντο · περισσότερη RAM
Τι γίνεται με πολύ μεγάλο κβάντο στον Round Robin;	γίνεται σαν FCFS · γίνεται σαν SJF · πολλές εναλλαγές

ΒΟΗΘΕΙΑ Οι ερωτήσεις του κουίζ. Αν κολλήσεις, ξαναδές τα **βήματα 1–7**.

Κουίζ επανάληψης: λύση

ΕΡΏΤΗΣΗ	ΣΩΣΤΉ ΑΠΆΝΤΗΣΗ	ΓΙΑΤΊ
Ποιος αλγόριθμος εξυπηρετεί με σειρά άφιξης;	FCFS	Πρώτος ήρθε, πρώτος εξυπηρετείται, όπως η ουρά στο ταμείο.
Ποιος δίνει τον μικρότερο μέσο χρόνο αναμονής χωρίς διακοπή;	SJF	Οι σύντομες μπαίνουν μπροστά. (Με διακοπή κερδίζει το SRTF.)
Τι είναι το κβάντο χρόνου;	ο χρόνος κάθε σειράς στον RR	Το **πολύ** τόσο κρατά τον επεξεργαστή μια διεργασία κάθε φορά.
Ποιοι αλγόριθμοι διακόπτουν;	RR, SRTF, Priority με διακοπή	Είναι οι **προεκχωρητικοί** του βήματος 1.
Τι είναι η λιμοκτονία;	δεν παίρνει ποτέ επεξεργαστή	Συμβαίνει στο SJF, στο SRTF και στο Priority.
Ποια λύση υπάρχει για τη λιμοκτονία;	η γήρανση (aging)	Όσο περιμένει, η προτεραιότητα **ανεβαίνει**.
Τι γίνεται με πολύ μεγάλο κβάντο στον Round Robin;	γίνεται σαν FCFS	Καμία δεν χρειάζεται δεύτερη σειρά (άσκηση 10).

ΘΥΜΗΣΟΥ Το **τεστ** έχει ερωτήσεις σαν αυτές και ένα Gantt με διακοπή.

Πολύ μεγάλο κβάντο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 10

ΕΚΦΩΝΗΣΗ

Τα δεδομένα της **άσκησης 3**, αλλά με κβάντο **4**. Λύσε το και σύγκρινε με το FCFS: τι παρατηρείς;

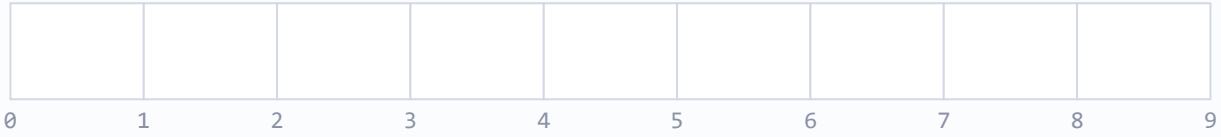
ΔΕΔΟΜΕΝΑ · ΚΒΑΝΤΟ = 4

ΔΙΕΡΓΑΣΊΑ	ΆΦΙΞΗ	ΕΚΤΈΛΕΣΗ
P1	0	4
P2	0	2
P3	0	3

Μέσος χρόνος αναμονής

(___ + ___ + ___) / 3 = ___

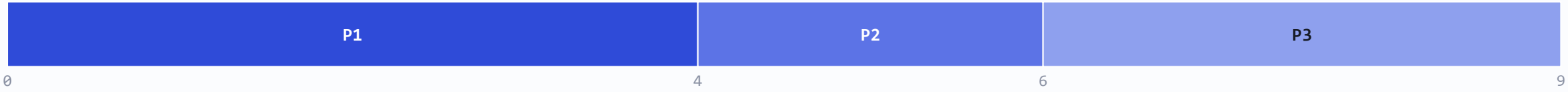
ΔΙΑΓΡΑΜΜΑ GANTT



ΔΙΕΡΓΑΣΊΑ	ΤΕΛΕΙΏΝΕΙ	ΑΝΑΜΟΝΉ
P1		
P2		
P3		

ΣΤΟ SITE Με κβάντο 4 **καμία** διεργασία δεν χρειάζεται δεύτερη σειρά.

Πολύ μεγάλο κβάντο: λύση



ΤΙ ΑΠΟΦΑΣΙΖΕΙ Ο ΧΡΟΝΟΔΡΟΜΟΛΟΓΗΤΗΣ ΚΑΘΕ ΦΟΡΑ ΠΟΥ ΚΑΤΙ ΑΛΛΑΖΕΙ

ΣΤΙΓΜΉ	ΟΥΡΆ (ΠΡΏΤΗ →)	ΤΡΈΧΕΙ	ΓΙΑΤΊ
0	P1, P2, P3	P1 (0–4)	Τρέχει η **P1**, η πρώτη της ουράς
4	P2, P3	P2 (4–6)	Τελείωσε η P1 → τρέχει η **P2**, η πρώτη της ουράς
6	P3	P3 (6–9)	Τελείωσε η P2 → τρέχει η **P3**, η πρώτη της ουράς

ΑΝΑΜΟΝΗ = ΤΕΛΟΣ − ΑΦΙΞΗ − ΕΚΤΕΛΕΣΗ

ΔΙΕΡΓ.	ΤΈΛΟΣ − ΆΦ. − ΕΚΤ.	ΑΝΑΜ.
P1	4 − 0 − 4	**0**
P2	6 − 0 − 2	**4**
P3	9 − 0 − 3	**6**

Μέσος όρος

(0 + 4 + 6) / 3 = 10 / 3 = **3,33**
Ίδιο ακριβώς με το **FCFS**. Με κβάντο 2 (άσκηση 3) ήταν 4.

ΘΥΜΗΣΟΥ Αν το κβάντο είναι ≥ από τη μεγαλύτερη εκτέλεση, ο Round Robin **γίνεται FCFS**.

Προτεραιότητα στο Linux

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 11 · 12

ΑΣΚΗΣΗ 11 · BASH

```
nice -n 10 gedit &      # ξεκίνα με nice 10
ps -o pid,ni,comm       # δες τη στήλη NI
ps -l                   # PRI = 80 + NI
nice -n -5 firefox &    # αρνητικό: μόνο ο root
```

Η κλίμακα nice

Από **−20** (πιο σημαντική) έως **19** (πιο «ευγενική»: αφήνει τις άλλες να περάσουν). Κανονικά **0**.
Είναι το **Priority** των ασκήσεων, σε αληθινό σύστημα.

ΑΣΚΗΣΗ 12 · BASH

```
pgrep paixnidi          # βρες το PID
renice -n 15 -p 2413    # νέο nice: 15
top                     # PR = 20 + NI
renice -n 0 -p 2413     # πίσω στο 0;
```

Ποιος μπορεί τι

Ο απλός χρήστης μπορεί μόνο να **ανεβάσει** το nice (να γίνει πιο ευγενικός). Αρνητικό nice ή επιστροφή προς τα κάτω: μόνο ο **root**. Αλλιώς: **Permission denied**.

ΣΤΟ SITE Το PID του paixnidi στον δικό σου εξομοιωτή θα είναι άλλο: βάλε **αυτό που σου δείχνει το pgrep**.

Τι γράφεις και τι βλέπεις

ΑΣΚΗΣΗ 11 · ΕΝΤΟΛΕΣ ($) ΚΑΙ ΕΞΟΔΟΣ

```
$ nice -n 10 gedit &
[1] 3196
$ ps -o pid,ni,comm
    PID  NI COMMAND
   3075   0 bash
   3196  10 gedit
$ ps -l                          # μόνο οι στήλες μας
    PID  PRI  NI CMD
   3196   90  10 gedit
$ nice -n -5 firefox &
nice: cannot set niceness: Permission denied
```

Απαντήσεις άσκησης 11

Το NI του gedit: **10** · το PRI: 80 + 10 = **90**
Αρνητικό nice δίνει μόνο ο **root** (διαχειριστής). Ο απλός χρήστης παίρνει **Permission denied**.

ΑΣΚΗΣΗ 12 · ΕΝΤΟΛΕΣ ($) ΚΑΙ ΕΞΟΔΟΣ

```
$ pgrep paixnidi
2413
$ renice -n 15 -p 2413
2413 (process ID) old priority 0, new priority 15
$ top
    PID USER      PR  NI ... %CPU COMMAND
   2413 mathitis  35  15 ... 99.7 paixnidi
   2075 mathitis  39  19 ...  0.3 tracker-miner
$ renice -n 0 -p 2413
renice: failed to set priority for 2413 … Permission denied
```

Απαντήσεις άσκησης 12

PR του paixnidi μετά το renice: 20 + 15 = **35**
Ήδη με NI **19**: το **tracker-miner** (ψάχνει αρχεία στο παρασκήνιο, δεν βιάζεται).
Πίσω στο 0 **δεν** σε αφήνει: χαμηλότερο nice δίνει μόνο ο **root** (sudo).

ΘΥΜΗΣΟΥ Στο Linux **μικρότερος** αριθμός = **πιο σημαντική**. Το nice αλλάζει το NI, και το PR/PRI ακολουθεί.

Προτεραιότητα στα Windows

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 13 · 14

ΑΣΚΗΣΕΙΣ 13 ΚΑΙ 14 · CMD

```
start /low notepad
start /high calc
wmic process where name="notepad.exe" get name,priority
wmic process get name,processid,priority
```

Ασκήσεις 13 και 14

Ποια προτεραιότητα έχει το Σημειωματάριο (/low) και ποια η Αριθμομηχανή (/high);
Ποια έχει το **csrss.exe** και ποια το **explorer.exe**; Τι γίνεται αν πας να αλλάξεις το **lsass.exe**;

	WINDOWS	LINUX
Κλίμακα	4 … 24	nice −20 … 19
Κανονική	**8** (Normal)	**0**
Πιο σημαντική	**μεγαλύτερος** αριθμός	**μικρότερος** αριθμός
Ξεκίνα με	start /low · /high	nice -n 10
Άλλαξε	wmic … setpriority	renice
Δεν επιτρέπεται	ReturnValue = 2	Permission denied

ΠΡΟΣΟΧΗ Οι δύο κλίμακες είναι **ανάποδες**! Στα Windows ο **μεγάλος** αριθμός είναι ο σημαντικός, στο Linux ο **μικρός**.

Τι γράφεις και τι βλέπεις

ΑΣΚΗΣΗ 13 · ΕΝΤΟΛΕΣ (>) ΚΑΙ ΕΞΟΔΟΣ

```
> start /low notepad
> start /high calc
> wmic process where name="notepad.exe" get name,priority
Name         Priority
notepad.exe  4
> wmic process where name="CalculatorApp.exe" get name,priority
Name               Priority
CalculatorApp.exe  13
```

Απαντήσεις άσκησης 13

Σημειωματάριο (/low): **4** · Αριθμομηχανή (/high): **13**
Στα Windows μεγαλύτερος αριθμός = **πιο σημαντική**.
Η Αριθμομηχανή λέγεται **CalculatorApp.exe** στη λίστα διεργασιών.

ΑΣΚΗΣΗ 14 · ΕΝΤΟΛΕΣ (>) ΚΑΙ ΕΞΟΔΟΣ

```
> wmic process get name,processid,priority
Name            Priority  ProcessId
csrss.exe       13        1448
explorer.exe    8         6172
chrome.exe      8         6672      # … και άλλες
> wmic process where name="csrss.exe" get name,priority
Name       Priority
csrss.exe  13
csrss.exe  13
> wmic process where name="lsass.exe" call setpriority "high priority"
Method execution failed: Access denied.
        ReturnValue = 2;
```

Απαντήσεις άσκησης 14

csrss.exe: **13** (διεργασία του ΛΣ: δεν πρέπει να «κολλήσει» ποτέ) · explorer.exe: **8** (κανονική)
lsass.exe: **ReturnValue = 2** (Access denied). Κρίσιμες διεργασίες του ΛΣ δεν αλλάζουν χωρίς διαχειριστή.

ΘΥΜΗΣΟΥ **ReturnValue = 0** σημαίνει ότι η αλλαγή έγινε. Οτιδήποτε άλλο είναι **λάθος** (το 2 = δεν έχεις δικαίωμα).

Πάγωσε και ταξινόμησε

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 15 · 16

ΑΣΚΗΣΗ 15 · BASH

```
killall -STOP firefox   # νάρκωση σε όλες
top                     # Tasks: … N stopped
killall -CONT firefox   # αφύπνιση
```

Μόνο οι έτοιμες παίζουν

Ο χρονοδρομολογητής διαλέγει **μόνο** ανάμεσα σε όσες είναι **έτοιμες**. Μια παγωμένη (κατάσταση **T**) μένει στη μνήμη αλλά δεν παίρνει επεξεργαστή μέχρι το **-CONT**.
Βρες: πόσες «stopped» δείχνει το top και ποιο σήμα τις πάγωσε.

ΑΣΚΗΣΗ 16 · BASH

```
ps -eo pid,ni,comm
ps -eo pid,ni,comm | sort -k2 -n | head -3
ps -eo pid,ni,comm | sort -k2 -nr | head -3
```

Ταξινόμηση κατά NI

-k2 -n: 2η στήλη (NI), από το μικρότερο (πιο σημαντική).
-r: ανάποδα, από το μεγαλύτερο (πιο ευγενική).
Βρες: την πιο σημαντική διεργασία, το NI της και την πιο «ευγενική».

ΘΥΜΗΣΟΥ Το **-STOP / -CONT** το είδαμε στο Εργαστήριο 2 (νάρκωση και αφύπνιση). Τώρα το βλέπουμε από τη μεριά του χρονοδρομολογητή.

Τι γράφεις και τι βλέπεις

ΑΣΚΗΣΗ 15 · ΕΝΤΟΛΕΣ ($) ΚΑΙ ΕΞΟΔΟΣ

```
$ killall -STOP firefox
$ top
Tasks: 214 total,   1 running, 206 sleeping,
       6 stopped,   0 zombie
$ killall -CONT firefox
```

Απαντήσεις άσκησης 15

Το top γράφει **6 stopped**: όσες firefox πάγωσες.
Τις πάγωσε το σήμα **SIGSTOP** (-STOP, αριθμός 19). Τις ξύπνησε το **SIGCONT**.
Όσο είναι παγωμένες **δεν** διεκδικούν τον επεξεργαστή: η ουρά των Έτοιμων μικραίνει.

ΑΣΚΗΣΗ 16 · ΕΝΤΟΛΕΣ ($) ΚΑΙ ΕΞΟΔΟΣ

```
$ ps -eo pid,ni,comm | sort -k2 -n | head -3
    397 -20 kworker/0:1H
      1   0 systemd
      2   0 kthreadd
$ ps -eo pid,ni,comm | sort -k2 -nr | head -3
   3244  19 tracker-miner
   4285   0 ps
   3923   0 bash
```

Απαντήσεις άσκησης 16

Πιο σημαντική: **kworker** (νήμα του πυρήνα) με NI **−20**.
Πιο «ευγενική»: **tracker-miner** με NI **19**.
Ο πυρήνας δίνει στον εαυτό του την **υψηλότερη** προτεραιότητα, οι δουλειές παρασκηνίου τη **χαμηλότερη**.

ΘΥΜΗΣΟΥ Στο **sort**: -k2 = 2η στήλη · -n = αριθμητικά (αλλιώς το «-20» δεν μπαίνει σωστά) · -r = ανάποδα.

Σύγκριση στο Excel και παραδοτέο

XRONODROMOLOGISI.XLSX · ΝΕΟ ΦΥΛΛΟ «ΣΥΓΚΡΙΣΗ»

```
A1: Διεργασία  B1: Άφιξη  C1: Εκτέλεση  D1: Τέλος
E2: =D2-B2-C2            # αναμονή (σύρε κάτω)
E6: =AVERAGE(E2:E5)      # μέσος όρος
```

4 λωρίδες χρόνου 0 … 16

Μία γραμμή για **FCFS, SJF, SRTF, Round Robin** με τα δεδομένα της άσκησης 5. Χρωμάτισε τα κελιά όπως στο Εργαστήριο 3.

Γράφημα

Επίλεξε τους 4 μέσους όρους → **Εισαγωγή** → **Γράφημα στηλών**. Ποια στήλη είναι η πιο χαμηλή;

#	ΠΑΡΑΔΟΤΈΟ: ΑΠΆΝΤΗΣΕ ΣΤΟ ΦΎΛΛΟ ΕΡΓΑΣΊΑΣ
1	Άσκηση 3 (Round Robin): μέσος χρόνος αναμονής
2	Άσκηση 5 (SRTF): μέσος χρόνος αναμονής
3	Άσκηση 6 (Priority): ποια διεργασία έτρεξε τελευταία;
4	Άσκηση 8: ποιος αλγόριθμος κέρδισε και με πόσο;
5	Γιατί τα Windows χρειάζονται αλγόριθμο με διακοπή;
6	Τι είναι η γήρανση και τι πρόβλημα λύνει;

ΑΝΕΒΑΣΕ το **Xronodromologisi.xlsx** με το φύλλο «Σύγκριση» και το γράφημα. Οι σωστοί μέσοι όροι είναι στη λύση της άσκησης 8.

Τι κρατάμε από σήμερα

Round Robin

Ένα **κβάντο** στην καθεμία, κυκλικά. **Δίκαιος**. Πολύ μεγάλο κβάντο = FCFS.

SRTF

Το SJF **με διακοπή**: τρέχει ο μικρότερος **υπόλοιπος** χρόνος. Μικρότερη μέση αναμονή.

Priority

Πρώτα η **πιο σημαντική**, με ή χωρίς διακοπή. Κίνδυνος **λιμοκτονίας**, λύση η **γήρανση**.

Αναμονή με διακοπή

Τέλος − άφιξη − εκτέλεση.
Χωρίς διακοπή αρκεί το αρχίζει − άφιξη.

Στο αληθινό ΛΣ

Linux: **nice** −20 … 19 (μικρό = σημαντικό). Windows: **4 … 24** (μεγάλο = σημαντικό).

Επόμενο εργαστήριο · Τρίτη 10/11

Πάμε στα **Windows**: MS-DOS και Γραμμή εντολών (cmd).

ΕΡΩΤΗΣΗ Ποια η διαφορά SJF και SRTF; Το SRTF **διακόπτει** όταν έρθει πιο σύντομη.